



docker build Command

Introduction

The **docker build** command is used to **create a Docker image** from a **Dockerfile**.

- What is **docker build** used for?
- How does it work?
- What are the different options for **docker build**?

This guide provides a **detailed explanation of docker build** with practical examples.

Section 1: Learn

What is **docker build**?

The **docker build** command **creates a Docker image** from a set of instructions in a **Dockerfile**.

Compon ent	Description
Dockerfil e	A text file with step-by-step instructions to create an image
Image	A snapshot of an application and its dependencies
Containe r	A running instance of an image



Why is **docker build** Important?

Feature	Benefit
Automates Image Creation	No need to manually configure dependencies
Ensures Consistency	Every container is created with the same setup
Speeds Up Deployment	Once built, images can be quickly deployed
Works Across Platforms	Runs the same way on any system

How **docker build** Works

1. Reads instructions from a **Dockerfile**.
2. Executes each step in the order listed.
3. Creates an image layer for each instruction.
4. Saves the final image in the local Docker registry.

Interesting Fact

Docker images are layered, meaning each command in a **Dockerfile** creates a new cached layer, making future builds faster.

Section 2: Practice

1. Writing a Simple Dockerfile

Create a file named **Dockerfile**:

```
# Use an official Python image as a base
FROM python:3.8
```



```
# Set the working directory inside the container
```

```
WORKDIR /app
```

```
# Copy project files into the container
```

```
COPY . /app
```

```
# Install dependencies
```

```
RUN pip install -r requirements.txt
```

```
# Define the command to run the app
```

```
CMD ["python", "app.py"]
```

2. Building a Docker Image

Run the following command in the same directory as the **Dockerfile**:

```
docker build -t my-python-app .
```

Flag	Meaning
-t my-python -app	Tags the image with a name
.	Specifies the current directory as the build context

To verify the image was built:

```
docker images
```



3. Running a Container from the Built Image

```
docker run -d -p 5000:5000 my-python-app
```

- Runs the `my-python-app` container.
- Maps port 5000 in the container to port 5000 on the host.

4. Using Build Arguments (`--build-arg`)

Modify the `Dockerfile` to accept a build argument:

```
ARG APP_ENV=development
```

```
# Print the environment variable during build
```

```
RUN echo "Environment: $APP_ENV"
```

To pass an argument during build:

```
docker build --build-arg APP_ENV=production -t my-app .
```

5. Caching and Optimizing Docker Builds

Docker caches layers to speed up builds.

To force a fresh build, use:

```
docker build --no-cache -t my-app .
```



Section 3: Know More

Frequently Asked Questions

1. What is the difference between `docker build` and `docker run`?
 - `docker build` creates an image.
 - `docker run` creates a container from an image.
2. Where does Docker store built images?
 - Locally, viewable with:

```
docker images
```

3. How do I delete an unused image?

```
docker rmi <image_id>
```

4. How do I tag an image with a version?

```
docker build -t my-app:v1 .
```

5. How do I push a built image to Docker Hub?

```
docker push my-dockerhub-username/my-app:v1
```

Conclusion

The `docker build` command creates Docker images from Dockerfiles, allowing repeatable, consistent, and efficient deployments.

Start by writing a Dockerfile, building an image, and running it as a container, and soon you'll be deploying applications effortlessly!